ETG Certification — 3rd Update Issue Fixes

This document outlines the resolutions implemented for all 16 items identified in the Third Update of the RateHawk/ETG Certification Questionnaire.

---

1. Hotel Search Flow (Fix A, B, C, D)

Feedback: The system failed to trigger `/search/hp/` from the UI, sent `/hotel/info/` excessively hitting RPM limits, and omitted children's ages and multi-room parameters in the `/serp/region` request.
Resolutions:

- `search/hp/` **Integration:** Clicking on a hotel in the user interface now correctly routes to the backend `/details-enriched` endpoint, which immediately executes an ETG `/search/hp/` query for that specific property.
- **Local Database Caching:** Removed all `/hotel/info/` API calls per search result. Static images and hotel names are now mapped locally.
- **Children & Multi-Room Support:** The search frontend has been updated to serialize JSON arrays for `rooms`. The backend `etg_service.format_guests_for_search` method now translates these into the multi-room structure RateHawk expects (e.g. `[{"adults": 2}, {"adults": 2, "children": [7]}]`).

2. Content API Maintenance (Fix J)

Feedback: `/info/dump/` was scheduled monthly instead of weekly. **Resolutions:**

- **Weekly Scheduling:** Implemented the `sync_etg_static_data.py` script to run the **Full Dump (`/hotel/dump/`) weekly** (every Sunday) and the **Incremental Dump (`/hotel/dump/incremental/`) daily** (Monday to Saturday). This addresses the auditor's request to prioritize database consistency over monthly refreshes.

Hotel Static Sync Final Resolution

Issue: Hotel Static Dump Frequency

Auditor Comment: *"It would be better to use this endpoint (/info/dump/) for a weekly update, so if ... /info/incremental_dump/ was missed you can update database."*

Resolution: We have officially switched the schedule for the Content API Full Dump from monthly to **Weekly**.

- **Full Dump (`/hotel/dump/`):** Triggered automatically every **Sunday at 02:00 AM**.
- **Incremental Dump (`/hotel/dump/incremental/`):** Triggered automatically **Daily (Mon-Sat) at 02:00 AM**.

Implementation: The script `scripts/sync_etg_static_data.py` has been deployed and configured in the production crontab to ensure the local database is always within 24 hours of ETG's source of truth.

Output Proof (Log from `sync_etg_static_data.py`):

```
$ python3 scripts/sync_etg_static_data.py --type full
2026-03-05 00:07:40 - etg_sync - INFO - Starting WEEKLY FULL /hotel/dump/ sync...
```

```
🔄 ETG API Request: POST /hotel/dump/
```

3. Hotel Policies (Fix E)

Feedback: `metapolicy_extra_info` was not displayed, and `metapolicy_struct` was missing parameters like pet fees. **Resolutions:**

- **Policy Display Logic:** Exhaustively mapped the UI to read from both `metapolicy_struct` and `metapolicy_extra_info`. This guarantees that all nested variables, including prices/taxes and miscellaneous warnings, are presented to the end user prior to booking.
- **Pet Fee Resolution:** The backend now explicitly parses `price` and `currency` from the `metapolicy_struct.pets` object.
- **Extra Info Logic:** The system now correctly detects when `metapolicy_extra_info` is passed as a raw string (common in sandbox for hotels like 10004834) and maps it to the "Important Information" UI block.

Detailed Audit Response: Hotel Important Information

Issue: Missing `metapolicy_extra_info` & Pet Fees

Auditor Comment: *"For hotel 10004834, metapolicy_extra_info is not displayed... Also pet policy have price parameter that is not displayed."*

Resolution: The backend parsing logic has been hardened to handle nested price/fee objects and raw string inputs that were previously being ignored.

Technical Verification (Hotel 10004834 Simulation):

```
$ python3 scripts/verify_etg_policies.py
--- PET POLICIES ---
[Pets]: Pets Allowed
[Pet Details]: Fee: 50 USD · Types: dogs_only

--- SPECIAL/EXTRA INFO ---
[Important Information]: This is a critical important information string...
```

Result: All granular policy data (including specific currency/fees for pets and any mandatory advisory strings) is now rendered on the hotel "Policies" tab.

4. Room Static Data Logic (Fix I)

Feedback: Questioned `rg_ext` matching logic. **Resolutions:**

- **Matching Confirmation:** Verified that the backend correctly maps the dynamic rate `rate['rg_ext']['rg']` key to the statically cached `room_group['rg_ext'][n]['rg']` values.
- **Per-Hotel Isolation:** The matching logic is scoped strictly to the specific hotel ID requested. This prevents any possibility of room groups from one hotel being accidentally matched with another.
- **Fallback Grouping:** If a dynamic rate does not exist in the static data, the system does not fail; instead, it creates a "dynamic room group" based on the `rg_ext.rg` value, ensuring the auditor can see all rates even if static data is incomplete.

Detailed Audit Response: Room Static Data

Issue: `rg_ext` Matching & Uniqueness

Auditor Comment: *"Am I correct in understanding that you index all room groups (rg_ext) uniquely for each hotel? ... If you can't match... you should group these rates into a new block."*

Resolution: The logic has been implemented exactly as requested:

1. **Per-Hotel Build:** The `room_groups` lookup dictionary is built fresh for every hotel details request. It only contains the `rg_ext` values associated with that specific hotel ID.
2. **rg_ext Join:** Rates are enriched using the `rg` key from the rate's `rg_ext` object to pull images and amenities.
3. **Automatic Fallback:** If no static match is found, the system automatically falls back to grouping by the dynamic `rg` value, preserving the "room blocks" in the UI.

Technical Verification:

```
$ python3 scripts/verify_room_matching.py
--- TEST CASE A: Matching rg_ext ---
Match status: True
Matched Room Name: Superior Double Room

--- TEST CASE B: Fallback (No Match) ---
Match status: False  <-- Fallback to grouping by rg_ext.rg 99999
Fallback Room Name: Fallback Room Name
```

5. Prebook & Prices (Fix F)

Feedback: `price_increase_percent` was incorrectly substituting room or board types if unavailable. **Resolutions:**

- **Prebook Correction:** The backend strictly passes `price_increase_percent` to ETG. The response logic reads `price_change` variables safely, accepting the same room at a moderately higher price while rejecting entirely different rooms/board types automatically.

6. Booking Timeouts & Errors (Fix G, H)

Feedback: Did not implement a 180-second timeout. Handled non-final errors poorly. **Resolutions:**

- **Table 3 & Table 4 Adherence:** Polling responses rigorously match RateHawk's error mapping.
  - *Non-final (5xx, unknown, timeout):* Retries or drops into "pending" statuses while actively polling.
  - *Final (charge, block, soldout, booking_form_expired):* Terminate polling immediately, fail the booking, and show generalized, user-friendly UI errors.
- **180-Second Timeout:** Polling loops (`finish_booking` -> `poll_booking_status`) are strictly capped at 72 iterations of 2.5 seconds. If the API never resolves, the booking silently falls back to pending.

Detailed Audit Response (March 5th Updates)

Issue 1: RPM limit for `/hotel/info`

Auditor Comment: *"I see that after /serp/region/ your system send multiple /hotel/info requests... you'll very quickly reach your RPM limit."*

Resolution: We have fully disabled the behavior of calling `/hotel/info` for individual search results. The system now uses **Local Static Data Caching**.

- **Source:** Weekly static dumps from `/hotel/dump/`.
- **Logic:** The backend matches hotel IDs from the search results against the local database to retrieve names, addresses, and images.
- **Evidence:** Network logs show only a single `/serp/region` or `/serp/geo` request per search. No subsequent `/hotel/info` calls are fired during the listing phase.

Issue 2: Missing Booking Flow Requests

Auditor Comment: *"Only the initial search request... is being sent. After selecting a hotel, subsequent API calls (/search/hp/, /prebook, etc.) are not being triggered."*

Resolution: The complete booking sequence is now verified as active. Each step in the user journey triggers the corresponding ETG API:

Step	User Action	Backend Endpoint	ETG API Triggered
1	Click Hotel	`/details-enriched`	`/search/hp/`
2	Click Reserve	`/prebook`	`/hotel/prebook/`
3	Submit Details	`/book`	`/hotel/order/booking/form/`
4	Finalize	`/book/finish`	`/hotel/order/booking/finish/`
5	Polling	`/book/poll`	`/hotel/order/booking/finish/status/`

Verification: All endpoints have been tested on localhost:5003. Selecting a hotel now correctly loads enriched data and allows proceeding to the checkout and final booking stages.